

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 9
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Поиск расстояний в графе»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.
Гуляевский Д.С.
Каташов К.А.

Приняли
Юрова О.В.
Деев М.В.

Пенза, 2025

Цель работы

Научиться искать расстояние между вершинами в графе.

Лабораторное задание

Задание 1

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++
3. *Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2*

1. *Реализуйте процедуру поиска расстояний на основе обхода в глубину
2. *Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности
3. *Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Описание метода решения задачи

Метод решения основан на поуровневом обходе графа от стартовой вершины, где на каждом шаге расстояние до новых вершин увеличивается на 1 относительно текущей вершины.

В основной функции `main` программа начинает с инициализации генератора случайных чисел и установки русской локализации. Пользователю предлагается ввести количество вершин в графе, после чего динамически выделяется память под матрицу смежности G и вектор расстояний `vis`. Матрица смежности заполняется случайными значениями: на главной диагонали устанавливаются нули (отсутствие петель), а для остальных элементов генерируются случайные значения 0 или 1, причем матрица делается симметричной для обеспечения неориентированности графа. Вектор расстояний инициализируется значением -1 для всех вершин, что маркирует их как непосещенные.

После генерации и вывода матрицы смежности на экран пользователь вводит стартовую вершину, с которой начинается обход. Затем вызывается функция

BFSD, которая реализует основной алгоритм поиска расстояний. В функции BFSD создается очередь *q*, куда помещается стартовая вершина, а ее расстояние устанавливается в 0. Далее в цикле пока очередь не пуста извлекается текущая вершина, выводятся ее данные и просматриваются все смежные с ней вершины. Для каждой смежной непосещенной вершины она добавляется в очередь, а ее расстояние вычисляется как расстояние до текущей вершины плюс 1. После завершения обхода выводятся расстояния от стартовой вершины до всех остальных вершин графа.

Этот метод гарантирует нахождение кратчайших расстояний в невзвешенном графе, так как обход в ширину посещает вершины в порядке возрастания их удаленности от стартовой точки. Алгоритм эффективно обрабатывает все достижимые вершины, обеспечивая корректное вычисление расстояний за время $O(V+E)$, где *V* - количество вершин, *E* - количество ребер.

Листинг

Задание 1(база)

```
#include <iostream>
#include <locale.h>
#include <time.h>
#include <cstdio>
#include <queue>
using namespace std;

void BFSD(int** G,int* dist, int numG, int s) {
    queue<int> q;
    int v;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {

            if (G[v][i] == 1 && dist[i] == -1) {
                q.push(i);
                dist[i] = dist[v]+1;
            }
        }
    }

    for (int i = 0; i < numG; i++) {
        printf("\nРасстояние от %d вершины до:", s);
        printf("\n%d:%d", i, dist[i]);
    }
}
```

```

    }
    return ;
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "rus");

    int** G;
    int* vis;
    int numG, s;

    std::cout << "Введите кол-во элементов: " << std::endl;
    std::cin >> numG;

    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    vis = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        vis[i] = -1;
        for (int j = 0; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d ", G[i][j]);
        }
        printf("\n");
    }
    printf("Введите нач вершину: ");
    scanf_s("%d", &s);
    printf("Порядок обхода: ");
    BFS(D, vis, numG, s);
    return 0;
}

```

Задание 1 с ++

```

#include <iostream>
#include <vector>
#include <queue>
#include <ctime>
#include <locale.h>

```

```
using namespace std;
```

```

void BFS_List(const vector<vector<int>>& G, vector<int>& dist, int s) {
    queue<int> q;
    dist[s] = 0;
    q.push(s);
}

```

```

cout << "\nПорядок обхода (BFS): ";

while (!q.empty()) {
    int v = q.front();
    q.pop();
    cout << v << " ";

    for (int u : G[v]) {
        if (dist[u] == -1) {
            dist[u] = dist[v] + 1;
            q.push(u);
        }
    }
}

cout << "\n\nРасстояния от вершины " << s << ":\n";
for (int i = 0; i < dist.size(); i++) {
    cout << i << ": " << dist[i] << endl;
}

}

int main() {
    setlocale(LC_ALL, "rus");
    srand(time(NULL));

    int numG;
    cout << "Введите количество вершин: ";
    cin >> numG;

    vector<vector<int>>> G(numG);    // списки смежности
    vector<vector<int>>> M(numG, vector<int>(numG, 0)); // матрица смежности
    vector<int> dist(numG, -1);

    // Генерация случайного неориентированного графа
    for (int i = 0; i < numG; i++) {
        for (int j = i + 1; j < numG; j++) {
            int edge = rand() % 2; // 0 или 1
            M[i][j] = M[j][i] = edge;

            if (edge == 1) {
                G[i].push_back(j);
                G[j].push_back(i);
            }
        }
    }

    // Вывод матрицы смежности
    cout << "\nМатрица смежности:\n";
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            cout << M[i][j] << " ";
        }
        cout << endl;
    }
}

```

```
// ¶ Вывод списков смежности
cout << "\nСписки смежности:\n";
for (int i = 0; i < numG; i++) {
    cout << i << ": ";
    for (int v : G[i]) {
        cout << v << " ";
    }
    cout << endl;
}

int s;
cout << "\nВведите начальную вершину: ";
cin >> s;

BFS_List(G, dist, s);

return 0;
}
```

Задание 2 с ++

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <ctime>
#include <chrono>
```

```
using namespace std;
```

```
// =====
// ===== DFS (списки) =====
// =====
```

```
void DFS_List(const vector<vector<int>>& G,
    vector<int>& dist,
    int v, int d,
    vector<int>& order)
{
    dist[v] = d;
    order.push_back(v);

    for (int u : G[v]) {
        if (dist[u] == -1) {
            DFS_List(G, dist, u, d + 1, order);
        }
    }
}
```

```
// =====
// ===== DFS (матрица) =====
// =====
```

```
void DFS_Matrix(const vector<vector<int>>& M,
    vector<int>& dist,
    int v, int d,
    vector<int>& order)
```

```

{
    dist[v] = d;
    order.push_back(v);

    int n = M.size();
    for (int u = 0; u < n; u++) {
        // Проверяем, есть ли ребро между v и u
        if (M[v][u] == 1 && dist[u] == -1) {
            DFS_Matrix(M, dist, u, d + 1, order);
        }
    }
}

// =====
// ===== BFS (списки) =====
// =====

void BFS_List(const vector<vector<int>>& G,
              vector<int>& dist,
              int s,
              vector<int>& order)
{
    queue<int> q;
    dist[s] = 0;
    q.push(s);

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        order.push_back(v);

        for (int u : G[v]) {
            if (dist[u] == -1) {
                dist[u] = dist[v] + 1;
                q.push(u);
            }
        }
    }
}

int main() {
    setlocale(LC_ALL, "rus");
    srand(time(NULL));

    int n;
    cout << "Введите количество вершин графа: ";
    cin >> n;

    // -----
    // Генерация графа
    // -----
    vector<vector<int>>> G(n);
    vector<vector<int>>> M(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {

```

```

for (int j = i + 1; j < n; j++) {
    int edge = rand() % 2;

    M[i][j] = M[j][i] = edge;

    if (edge == 1) {
        G[i].push_back(j);
        G[j].push_back(i);
    }
}
}

// -----
// Вывод матрицы смежности с нумерацией
// -----
cout << "\nМатрица смежности:\n ";
for (int j = 0; j < n; j++)
    cout << j << " ";
cout << "\n";

for (int i = 0; i < n; i++) {
    cout << i << " | ";
    for (int j = 0; j < n; j++)
        cout << M[i][j] << " ";
    cout << endl;
}

// -----
// Вывод списков смежности
// -----
cout << "\nСписки смежности:\n";
for (int i = 0; i < n; i++) {
    cout << i << ": ";
    for (int v : G[i])
        cout << v << " ";
    cout << endl;
}

int start;
cout << "\nВведите начальную вершину: ";
cin >> start;

// -----
// DFS через списки смежности
// -----
vector<int> distDFS_list(n, -1);
vector<int> orderDFS_list;

auto t1 = chrono::high_resolution_clock::now();
DFS_List(G, distDFS_list, start, 0, orderDFS_list);
auto t2 = chrono::high_resolution_clock::now();

double timeDFS_list =
    chrono::duration<double, micro>(t2 - t1).count();

```



```

// -----
// DFS через матрицу смежности
// -----
vector<int> distDFS_matrix(n, -1);
vector<int> orderDFS_matrix;

auto t5 = chrono::high_resolution_clock::now();
DFS_Matrix(M, distDFS_matrix, start, 0, orderDFS_matrix);
auto t6 = chrono::high_resolution_clock::now();

double timeDFS_matrix =
    chrono::duration<double, micro>(t6 - t5).count();

// -----
// BFS
// -----
vector<int> distBFS(n, -1);
vector<int> orderBFS;

auto t3 = chrono::high_resolution_clock::now();
BFS_List(G, distBFS, start, orderBFS);
auto t4 = chrono::high_resolution_clock::now();

double timeBFS =
    chrono::duration<double, micro>(t4 - t3).count();

// -----
// Вывод результатов
// -----

cout << "\n===== DFS (списки) =====\n";
cout << "Порядок обхода: ";
for (int v : orderDFS_list) cout << v << " ";
cout << "\nРасстояния:\n";
for (int i = 0; i < n; i++) cout << i << ": " << distDFS_list[i] << endl;
cout << "Время выполнения DFS (списки): " << timeDFS_list << " мкс\n";

cout << "\n===== DFS (матрица) =====\n";
cout << "Порядок обхода: ";
for (int v : orderDFS_matrix) cout << v << " ";
cout << "\nРасстояния:\n";
for (int i = 0; i < n; i++) cout << i << ": " << distDFS_matrix[i] << endl;
cout << "Время выполнения DFS (матрица): " << timeDFS_matrix << " мкс\n";

cout << "\n===== BFS (списки) =====\n";
cout << "Порядок обхода: ";
for (int v : orderBFS) cout << v << " ";
cout << "\nРасстояния:\n";
for (int i = 0; i < n; i++) cout << i << ": " << distBFS[i] << endl;
cout << "Время выполнения BFS: " << timeBFS << " мкс\n";

cout << "\n===== \n";
cout << "Сравнение времени:\n";
cout << "DFS (списки): " << timeDFS_list << " мкс\n";

```

```
cout << "DFS (матрица): " << timeDFS_matrix << " мкс\n";  
cout << "BFS (списки): " << timeBFS << " мкс\n";  
  
return 0;  
}
```

Результаты работы программы

Задание 1-2

```

Консоль отладки Microsoft Visual Studio
Введите кол-во элементов:
7
0 1 0 0 1 1 0
1 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 1
1 0 0 0 0 0 1
1 0 0 0 0 0 0
0 0 0 1 1 0 0
Введите нач вершину: 0
Порядок обхода: 0 1 4 5 6 3
Расстояние от 0 вершины до:
0:0
Расстояние от 0 вершины до:
1:1
Расстояние от 0 вершины до:
2:-1
Расстояние от 0 вершины до:
3:3
Расстояние от 0 вершины до:
4:1
Расстояние от 0 вершины до:
5:1
Расстояние от 0 вершины до:
6:2
E:\rep\lab9\lab9.1\x64\Debug\lab9.1.exe (процесс 5448) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

```

```

Консоль отладки Microsoft Visual Studio
Введите количество вершин: 5
Матрица смежности:
0 1 0 1 0
1 0 0 0 0
0 0 0 0 1
1 0 0 0 1
0 0 1 1 0
Списки смежности:
0: 1 3
1: 0
2: 4
3: 0 4
4: 2 3
Введите начальную вершину: 0
Порядок обхода (BFS): 0 1 3 4 2
Расстояния от вершины 0:
0: 0
1: 1
2: 3
3: 1
4: 2

```

Консоль отладки Microsoft Visual Studio

Введите количество вершин графа: 5

Матрица смежности:

```

    0 1 2 3 4
0 | 0 1 1 1 0
1 | 1 0 1 1 1
2 | 1 1 0 0 0
3 | 1 1 0 0 0
4 | 0 1 0 0 0

```

Списки смежности:

```

0: 1 2 3
1: 0 2 3 4
2: 0 1
3: 0 1
4: 1

```

Введите начальную вершину: 0

===== DFS (списки) =====

Порядок обхода: 0 1 2 3 4

Расстояния:

```

0: 0
1: 1
2: 2
3: 2
4: 2

```

Время выполнения DFS (списки): 6.6 мкс

===== DFS (матрица) =====

Порядок обхода: 0 1 2 3 4

Расстояния:

```

0: 0
1: 1
2: 2
3: 2
4: 2

```

Время выполнения DFS (матрица): 5.1 мкс

===== BFS (списки) =====

Порядок обхода: 0 1 2 3 4

Расстояния:

```

0: 0
1: 1
2: 1
3: 1
4: 2

```

Время выполнения BFS: 12.4 мкс

=====

Сравнение времени:

```

DFS (списки): 6.6 мкс
DFS (матрица): 5.1 мкс
BFS (списки): 12.4 мкс

```

E:\ger\9.3\x64\Debug\9.3.exe (процесс 10400) завершил работу с кодом 0 (0x0).

Нажмите любую клавишу, чтобы закрыть это окно:

Выводы

В ходе выполнения данной лабораторной работы были успешно освоены методы поиска расстояний в графе с использованием обхода в ширину (BFS). Была реализована программа на языке C++, которая формирует матрицу смежности для неориентированного графа на основе генератора случайных чисел, после чего выполняет поиск расстояний от заданной стартовой вершины до всех остальных вершин. Алгоритм корректно обрабатывает как связанные, так и несвязанные вершины, отмечая недостижимые из начальной вершины значениями -1. В процессе работы было подтверждено, что обход в ширину позволяет находить кратчайшие расстояния в невзвешенном графе, поскольку посещение вершин происходит по уровням, начиная от стартовой вершины. Данный метод демонстрирует высокую эффективность для разреженных графов и прост в реализации с использованием очереди. В дальнейшем можно расширить функционал программы, добавив поддержку ориентированных графов, взвешенных рёбер, а также реализовав алгоритмы на основе обхода в глубину для сравнения производительности и анализа различных подходов к решению задачи поиска расстояний в графах.